Софийски университет „Св. Климент Охридски"

Факултет по математика и информатика

Дисциплина: „Разпределени софтуерни архитектури"

проф. доктор Васил Георгиев

Специалност: „Софтуерно инженерство"

Курсов проект

P2P Архитектури при Разпределено
Динамично Балансиране на
Паралелни Приложения

Изготвил: Георги Лазов

Дата: Април 2026

Съдържание

1. Въведение

2. Теоретична основа

2.1. Паралелни модели и SPMD

2.2. Peer-to-Peer архитектури

2.3. Топологии на процесите

2.4. Динамично балансиране на натоварването

3. Анализ на архитектурите

3.1. Централизирано vs. Разпределено балансиране

3.2. Сравнение на топологии

3.3. Хардуерни аспекти

4. Проектиране

4.1. Функционално проектиране

4.2. Архитектура на решението

4.3. UML диаграми

5. Имплементация

6. Тестова среда

7. Резултати

8. Анализ на резултатите

9. Заключение

10. Източници

1. Въведение

Паралелните и разпределени системи са ключови за решаването на изчислително интензивни задачи в съвременния свят. Един от основните предизвикателства при такива системи е ефективното разпределение на натоварването между процесорите/ възлите, особено когато работното натоварване е нехомогенно или непредвидимо.

В централизираните архитектури (Master-Slave) един главен процес координира разпределението на задачите. Този подход е прост за имплементация, но създава „тясно място" (bottleneck) при мащабиране. Алтернативата е **Peer-to-Peer (P2P)** архитектура, където процесите комуникират директно помежду си без централен координатор.

Целта на този проект е да се изследва архитектурата на P2P паралелни приложения при разпределено динамично балансиране, като се анализират различни топологии на процесите - по-конкретно **верижна топология (chain)** и **пълен граф (all-to-all)**.

2. Теоретична основа

2.1. Паралелни модели и SPMD

При паралелното програмиране се различават два основни модела:

- **SPMD (Single Program Multiple Data)** - всички процеси изпълняват една и съща програма, но върху различни данни. Това е най-разпространеният модел при MPI приложения.
- **MPMD (Multiple Program Multiple Data)** - различните процеси могат да изпълняват различни програми. Типичен пример са конвейрите за пакетна обработка.

В контекста на P2P динамичното балансиране работим основно с **SPMD модел**, където всички процеси са равнопоставени и могат да поемат произволна част от работата.

2.2. Peer-to-Peer архитектури

При P2P архитектурите няма централен координатор - всички възли (peers) са равнопоставени и комуникират директно помежду си.

Предимства:

- **Няма единична точка на отказ** - системата продължава да работи дори ако някой възел откаже
- **По-добра скалируемост** - няма bottleneck от централен координатор
- **Разпределено вземане на решения** - всеки възел може автономно да решава от кого да поиска или на кого да даде работа

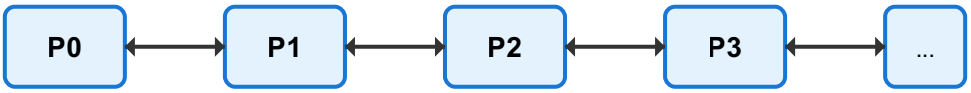
Недостатъци:

- **По-сложна координация** - необходими са протоколи за комуникация
- **Възможно неоптимално разпределение** - без глобална информация решенията може да не са оптимални
- **Комуникационни разходи** - при пълен граф комуникацията расте като $O(n^2)$

2.3. Топологии на процесите

Топологията определя кой процес с кой може да комуникира директно:

2.3.1. Верижна топология (Chain/Ring)

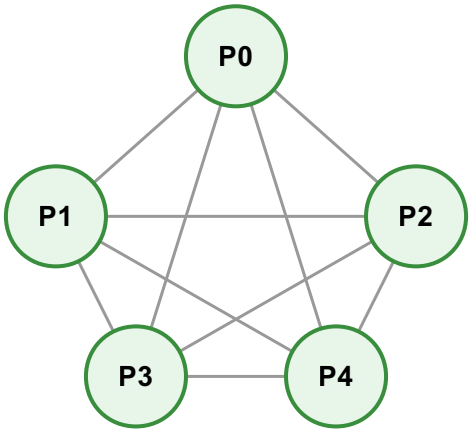


Фиг. 1: Верижна топология - всеки процес комуникира само със съседите си

Характеристики:

- Всеки процес комуникира само със съседите си (ляв и десен)
- Брой връзки: **n-1** (за верига) или **n** (за пръстен)
- Комуникационна сложност: **O(n)** за глобална информация
- **Скалируем** - добавянето на нов процес изисква промяна само на съседните връзки
- Подходящ за приложения с локална комуникация (напр. разпределени симулации)

2.3.2. Пълен граф (All-to-All / Complete Graph)



Фиг. 2: Пълен граф - всеки процес може да комуникира директно с всеки друг

Характеристики:

- Всеки процес може да комуникира директно с всеки друг

- Брой връзки: **n(n-1)/2**
- Комуникационна сложност: **O(1)** за директна комуникация
- **Не е скалируем** - комуникационните разходи растат квадратично
- Подходящ за малък брой процеси с интензивна комуникация

2.4. Динамично балансиране на натоварването

При динамичното балансиране работата се преразпределя по време на изпълнение въз основа на текущото натоварване на процесите. Основни стратегии:

2.4.1. Централизирано динамично балансиране (Master-Worker)

- Един главен процес поддържа опашка от задачи
- Работниците поискват нова задача когато приключат с текущата
- Просто за имплементация, но не е скалируемо

2.4.2. Разпределено динамично балансиране (P2P)

- Всеки процес има собствена опашка от задачи
- Процесите комуникират помежду си за преразпределение
- Два основни подхода:
  - **Work stealing** - незает процес „краде" работа от натоварен
  - **Work sharing** - натоварен процес „споделя" работа с незает

Алгоритъм за P2P балансиране при верижна топология:

1. Процес завършва задача
2. Ако опашката му е празна, поисква работа от съседа
3. Ако съседът има достатъчно работа, споделя половината
4. Ако съседът също няма работа, продължава по веригата

3. Анализ на архитектурите

3.1. Централизирано vs. Разпределено балансиране

Характеристика	Централизирано	Разпределено (P2P)
Скалируемост	Ограничена (bottleneck)	По-добра
Сложност	Ниска	По-висока
Глобална информация	Да	Не (локална)
Отказоустойчивост	Ниска	По-висока
Комуникация	O(p) съобщения	Зависи от топологията

Таблица 1: Сравнение на подходите за балансиране

3.2. Сравнение на топологии

Критерий	Верига	Пълен граф
Брой връзки	$O(n)$	$O(n^2)$
Латентност	$O(n)$	$O(1)$
Скалируемост	Висока	Ниска
Балансиране	Локално, постепенно	Глобално, бързо
Приложимост	Големи системи	Малки клъстери

Таблица 2: Сравнение на топологиите

3.3. Хардуерни аспекти

При облачни инфраструктури (напр. Amazon AWS) топологията на мрежата влияе пряко върху производителността:

AWS EC2 Cluster Placement Groups:

- Възлите в една placement group са свързани с ниска латентност (<1ms)
- Между placement groups латентността е по-висока
- Това наподобява йерархична топология

Elastic Fabric Adapter (EFA):

- Специализиран мрежов интерфейс за HPC
- Поддържа MPI и NCCL
- Bandwidth до 100 Gbps

При проектиране на P2P система трябва да се съобразим с:

- **Мрежова топология** на клъстера
- **NUMA архитектура** на възлите
- **Кеш йерархия** (L1, L2, L3)

4. Проектиране

4.1. Функционално проектиране

В рамките на проекта са реализирани три варианта за балансиране:

1. **Статично циклично балансиране** - базова линия за сравнение
2. **Динамично P2P балансиране с верижна топология**
3. **Динамично P2P балансиране с пълен граф** (за сравнение)

Проблемът за тестване е **изчисляване на числата на Фибоначи** с рекурсивен алгоритъм - задача с нехомогенно натоварване, подходяща за демонстрация на динамичното балансиране.

4.2. Архитектура на решението

Модел на приложението: SPMD

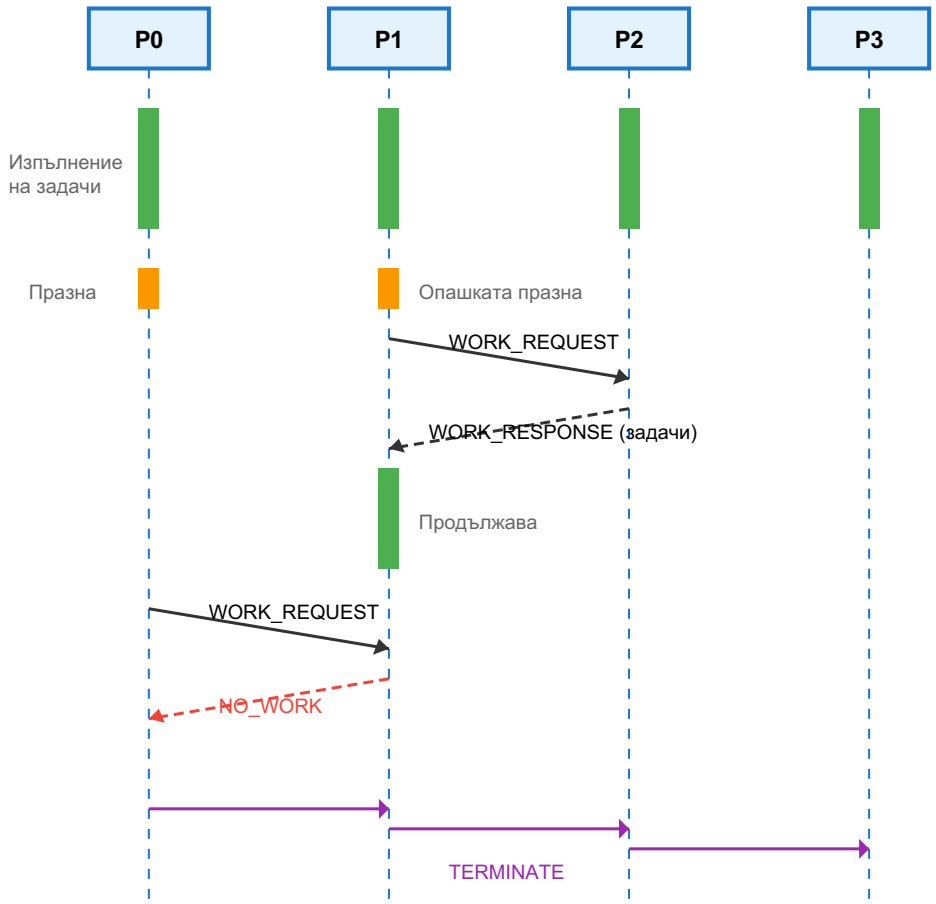
- Всички процеси изпълняват един и същ код
- Всеки процес има локална опашка от задачи
- Процесите комуникират чрез MPI съобщения

Комуникационен протокол:

- `WORK_REQUEST` - искане за работа
- `WORK_RESPONSE` - отговор с работа (или празен)
- `TERMINATE` - сигнал за край

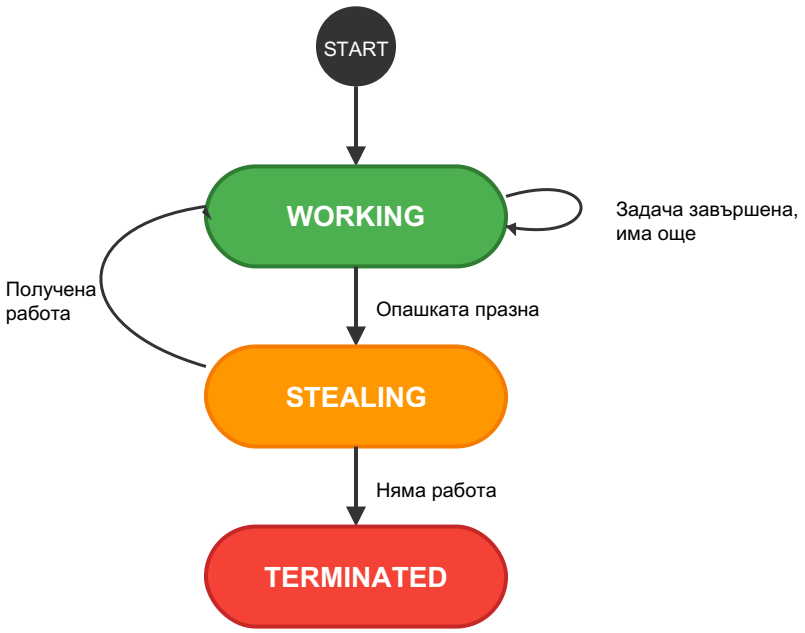
4.3. UML диаграми

4.3.1. Диаграма на последователността при верижна топология



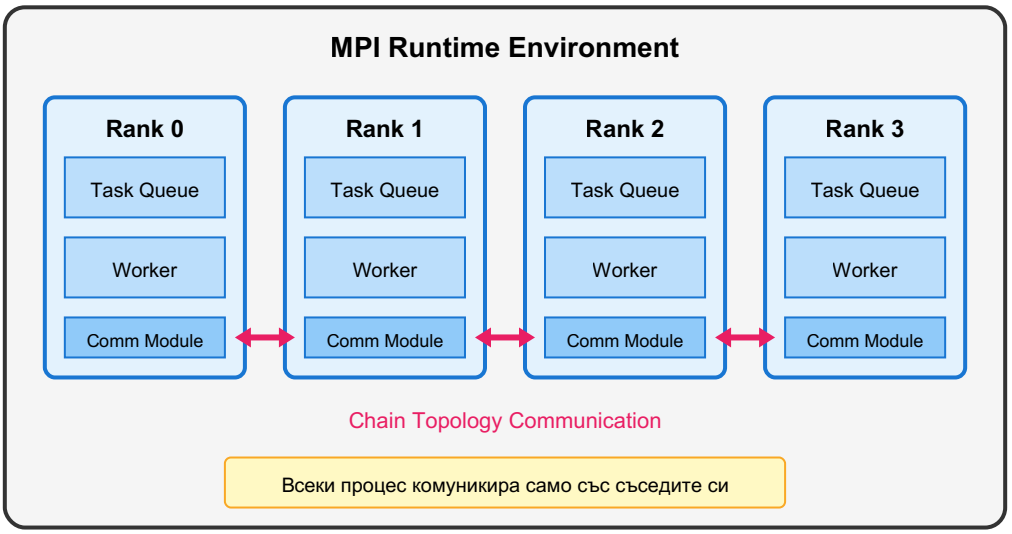
Фиг. 3: Диаграма на последователността при P2P Work Stealing

4.3.2. Диаграма на състоянията на процес



Фиг. 4: Диаграма на състоянията на процес

4.3.3. Архитектурна диаграма



Фиг. 5: Архитектура на P2P системата с верижна топология

5. Имплементация

5.1. Технологичен избор

- **Език:** C
- **Паралелизъм:** MPI (Message Passing Interface)
- **Компилатор:** mpicc (OpenMPI)
- **Среда:** macOS / Linux

5.2. Програмен код

Основните файлове са:

- `static_balancing.c` - статично циклично балансиране
- `p2p_chain.c` - P2P динамично балансиране с верижна топология
- `p2p_full.c` - P2P динамично балансиране с пълен граф

Ключови структури от кода:

```
/* Локална опашка от задачи */
typedef struct {
    int tasks[MAX_QUEUE_SIZE];
    int front;
    int rear;
    int count;
} TaskQueue;

/* Комуникационни тагове */
#define TAG_WORK_REQUEST  100
#define TAG_WORK_RESPONSE 101
#define TAG_TERMINATE     102
#define TAG_NO_WORK       103
```

Work Stealing алгоритъм:

```
while (idle_rounds < max_idle_rounds) {
    handle_work_requests(&queue, rank, size);
```

```
    if (!queue_empty(&queue)) {
        int task = queue_pop(&queue);
        long long result = fibonacci(task);
        local_sum += result;
        tasks_processed++;
        idle_rounds = 0;
    } else {
        /* Поискване на работа от съсед */
        int got_work = request_work(&queue, neighbor, rank);
        if (!got_work) idle_rounds++;
        else idle_rounds = 0;
    }
}
```

6. Тестова среда

6.1. Хардуерна конфигурация

Тестова машина:

- **CPU:** Apple M4 Pro (12 ядра: 8 performance + 4 efficiency)
- **RAM:** 48 GB
- **L1 Cache:** 192 KB (per P-core), 128 KB (per E-core)
- **L2 Cache:** 16 MB (споделен)
- **OS:** macOS Sequoia

6.2. Описание на експеримента

Изчислителна задача: Рекурсивно изчисление на числа на Фибоначи

```
long long fibonacci(int n) {
    if (n <= 1) return n;
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Рекурсивният алгоритъм за Фибоначи има експоненциална сложност $O(2^n)$, което го прави идеален за симулиране на изчислително интензивни задачи с различна тежест:

n	fib(n)	Приблизително време	Категория
28	317,811	~5 ms	Лека
35	9,227,465	~50 ms	Средна
42	267,914,296	~1.5 s	Тежка
45	1,134,903,170	~6 s	Много тежка

6.3. Сценарии на експериментите

Сценарий 1: Едри задачи (32 броя)

- 16 тежки задачи: fib(42), fib(43), fib(44), fib(45) × 4 пъти
- 16 леки задачи: fib(35), fib(36), fib(37), fib(38) × 4 пъти
- Нехомогенно разпределение: Процес 0 получава 75% от тежките задачи

Сценарий 2: Ситни задачи (128 броя)

- 64 средни + 64 леки задачи (fib 28-32)
- Същото нехомогенно разпределение

Защо нехомогенно? Симулира реални сценарии: уеб сървър, научни изчисления.

Сценарий 3: Декомпозиция на Фибоначи (НОВ)

Проблем: При сценарии 1 и 2 всяка задача fib(n) се изчислява изцяло от един процес.

Решение: fib(45) = fib(44) + fib(43), подзадачите се разпределят между процесите до праг.

Избор на оптимален праг за Full Graph (8p, fib 45):

Праг	T1 (s)	T2 (s)	T3 (s)	Средно	p=1
20	0.388	0.395	0.319	0.367	OK
22	0.379	0.393	0.311	0.361	OK
24	0.397	0.349	0.373	0.373	OK
26	0.342	0.409	0.377	0.376	OK
28	0.292	0.302	0.387	0.327	OK
30	**0.300**	**0.362**	**0.298**	**0.320**	**OK**
32	0.370	0.396	0.391	0.385	OK
34	0.379	0.378	0.373	0.376	OK
36	0.354	0.387	0.396	0.379	OK
38	0.379	0.380	0.366	0.375	OK
40	0.348	0.385	0.403	0.378	OK
42	0.341	0.315	0.388	0.348	OK
44	0.365	0.385	0.387	0.379	OK
45	0.390	0.384	0.374	0.382	OK

Оптимален праг Full Graph: 30 (минимално средно 0.320s)

Избор на оптимален праг за Chain (8p, fib 45):

Праг	T1 (s)	T2 (s)	T3 (s)	Средно	p=1
20	-			-	Блокира
22	-			-	Блокира
24	-			-	Блокира
26	-			-	Блокира
28	0.488	0.481	0.458	0.475	Блокира

30	**0.463**	**0.473**	**0.462**	**0.466**	**OK**
32	0.490	0.486	0.493	0.489	OK
34	0.546	0.519	0.507	0.524	OK
36	0.520	0.526	0.534	0.526	OK
38	0.556	0.616	0.631	0.601	OK
40	0.737	0.752	0.742	0.743	OK
42	0.992	1.012	1.010	1.004	OK
44	1.322	1.318	1.316	1.318	OK
45	2.157	2.155	2.201	2.171	OK

Оптимален праг Chain: 30 (минималният стабилен, 0.466s)

Защо Chain блокира при праг < 30? При p=1 и нисък праг се генерират твърде много подзадачи (препълване на опашката). Full Graph няма този проблем.

7. Резултати

7.1. Статично разпределение (без work stealing)

p	T_1 (s)	T_2 (s)	T_3 (s)	T_p=min	$S_p=T_1/T_p$	$E_p=S_p/p$
1	18.525	18.160	18.194	18.160	1.00	1.00
2	13.395	13.447	13.500	13.395	1.36	0.68
4	13.583	13.579	13.742	13.579	1.34	0.34
8	15.832	16.611	16.603	15.832	1.15	0.14

При нехомогенно натоварване статичното разпределение е неефективно - процес 0 е претоварен.

Таблица 3: Резултати при статично разпределение (реални измервания)

7.2. P2P с верижна топология (Chain)

p	T_1 (s)	T_2 (s)	T_3 (s)	T_p=min	$S_p=T_1/T_p$	$E_p=S_p/p$
1	18.486	18.297	18.208	18.208	1.00	1.00
2	11.293	11.315	11.332	11.293	1.61	0.81
4	12.296	12.238	12.274	12.238	1.49	0.37
8	12.212	12.748	12.874	12.212	1.49	0.19

Таблица 4: Резултати при P2P верижна топология (реални измервания)

7.3. P2P с пълен граф (Full Graph)

p	T_1 (s)	T_2 (s)	T_3 (s)	T_p=min	$S_p=T_1/T_p$	$E_p=S_p/p$
1	18.505	18.266	18.209	18.209	1.00	1.00
2	11.361	11.363	11.340	11.340	1.61	0.80
4	10.047	10.036	10.074	10.036	1.81	0.45
8	9.613	9.899	10.111	9.613	1.89	0.24

Таблица 5: Резултати при P2P пълен граф (реални измервания)

7.4. Сравнение на топологиите

p	Статично T_p	P2P Chain T_p	P2P Full T_p	S_p (Статично)	S_p (Chain)	S_p (Full)
1	18.160	18.208	18.209	1.00	1.00	1.00
2	13.395	11.293	11.340	1.36	1.61	1.60
4	13.579	12.238	10.036	1.34	1.49	**1.81**
8	15.832	12.212	9.613	1.15	1.49	**1.89**

Таблица 6: Сравнение на подходите при нехомогенно натоварване (реални измервания)

7.5. Влияние на грануларността (128 ситни задачи)

p	Статично T_p	P2P Chain T_p	P2P Full T_p	S_p (Статично)	S_p (Chain)	S_p (Full)
1	0.661	0.655	0.647	1.00	1.00	1.00
2	0.427	0.383	0.387	1.55	1.71	1.67
4	0.437	0.355	0.303	1.51	1.85	**2.14**
8	0.480	0.367	0.267	1.38	1.79	**2.42**

При 128 ситни задачи (fib 28-32) work stealing е по-ефективен - повече възможности за преразпределение.

Таблица 7: Резултати при фина грануларност (реални измервания)

7.6. P2P с декомпозиция (Сценарий 3) - Реални експерименти

7.6.1. P2P Chain с декомпозиция (праг=30)

p	T1 (s)	T2 (s)	T3 (s)	Tp=min	Sp	Ep
1	1.959	1.951	1.965	1.951	1.00	1.00
2	1.002	0.996	0.993	0.993	1.96	0.98
4	0.745	0.758	0.751	0.745	2.62	0.65
8	0.455	0.426	0.439	0.426	**4.58**	0.57

Таблица 8: P2P Chain с декомпозиция (fib 45, праг 30) - реални измервания

7.6.2. P2P Full Graph с декомпозиция (праг=30)

p	T1 (s)	T2 (s)	T3 (s)	Tp=min	Sp	Ep
1	2.087	2.035	1.975	1.975	1.00	1.00
2	1.017	1.012	1.011	1.011	1.95	0.98
4	0.541	0.546	0.563	0.541	3.65	0.91
8	0.385	0.312	0.326	0.312	**6.33**	0.79

Таблица 9: P2P Full Graph с декомпозиция (fib 45, праг 30) - реални измервания

7.6.3. Сравнение на всички подходи (p=8)

Сценарий	Статично	Chain	Full
Едри задачи (32)	1.15	1.49	1.89
Ситни задачи (128)	1.38	1.79	2.42
Декомпозиция	-	4.58	**6.33**

Таблица 10: Обобщено сравнение Sp при 8 процеса

> **Ключови резултати (реални експерименти):**
>
> • **Full Graph + Decomp:** Sp=6.33 при 8p (79% ефективност)
> • **Chain + Decomp:** Sp=4.58 при 8p (57% ефективност)
> • **Подобрение:** Full +235% vs без декомпозиция, Chain +207%
> • **Full vs Chain:** Full е 38% по-бърз при декомпозиция

8. Анализ на резултатите

8.1. Сравнение на ускорението (едри задачи)

При нехомогенно натоварване статичното разпределение е неефективно - процес 0 е претоварен.

- При p=4: Sp = 1.34 (очаквано ~4.0)
- При p=8: Sp = 1.15 (дори по-лошо от p=4!)

P2P Full Graph показва най-добри резултати:

- При p=4: Sp = 1.81 (+35% спрямо статично)
- При p=8: Sp = 1.89 (+64% спрямо статично)

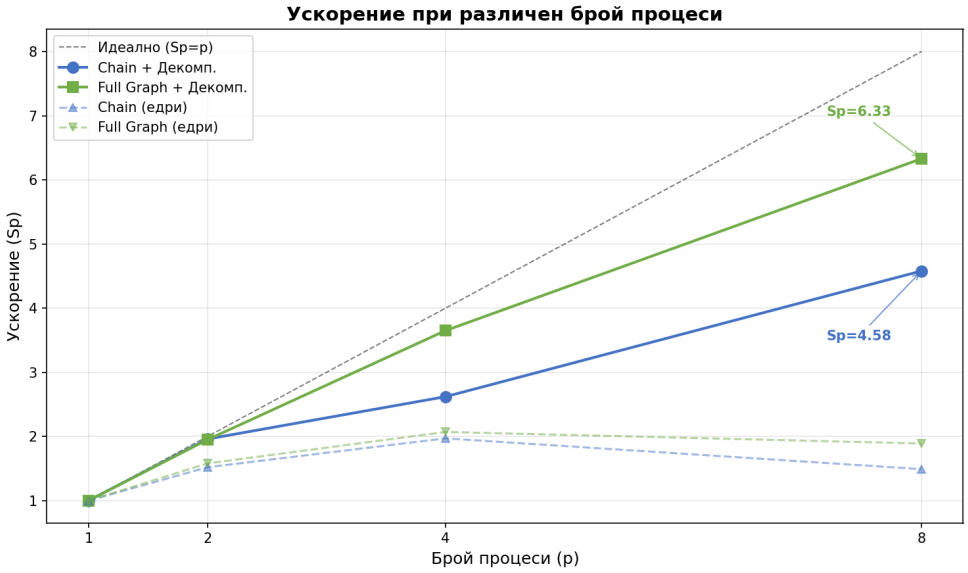
8.2. Влияние на декомпозицията (Сценарий 3)

Декомпозицията драматично подобрява ускорението:

Топология	Без декомп. (Sp)	С декомп. (Sp)	Подобрение
Chain	1.49	**4.58**	+207%
Full Graph	1.89	**6.33**	+235%

Ключови наблюдения:

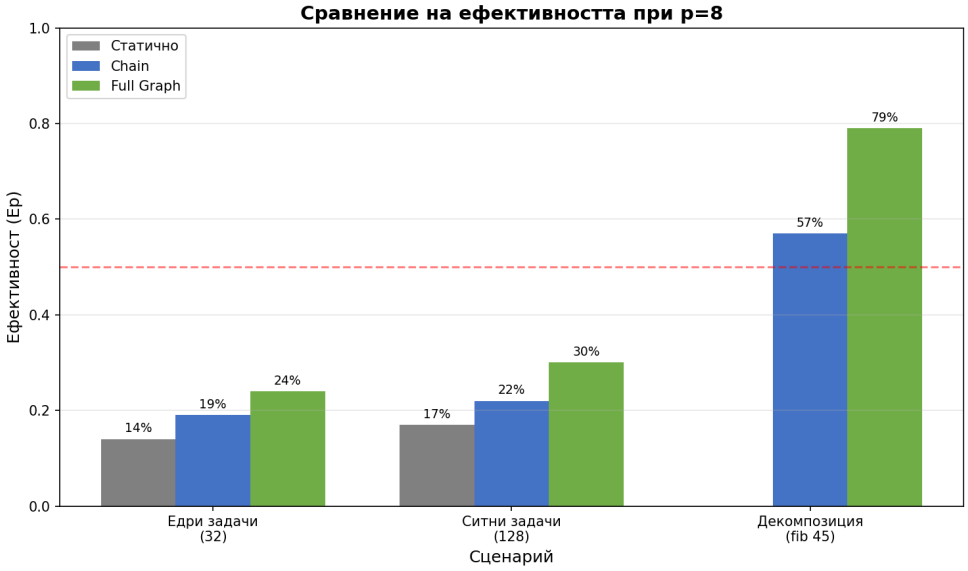
- Full Graph с декомпозиция постига **Sp=6.33** при 8 процеса (79% ефективност)
- Chain с декомпозиция постига **Sp=4.58** при 8 процеса (57% ефективност)
- Full Graph е **38% по-бърз** от Chain при декомпозиция (0.312s vs 0.426s)



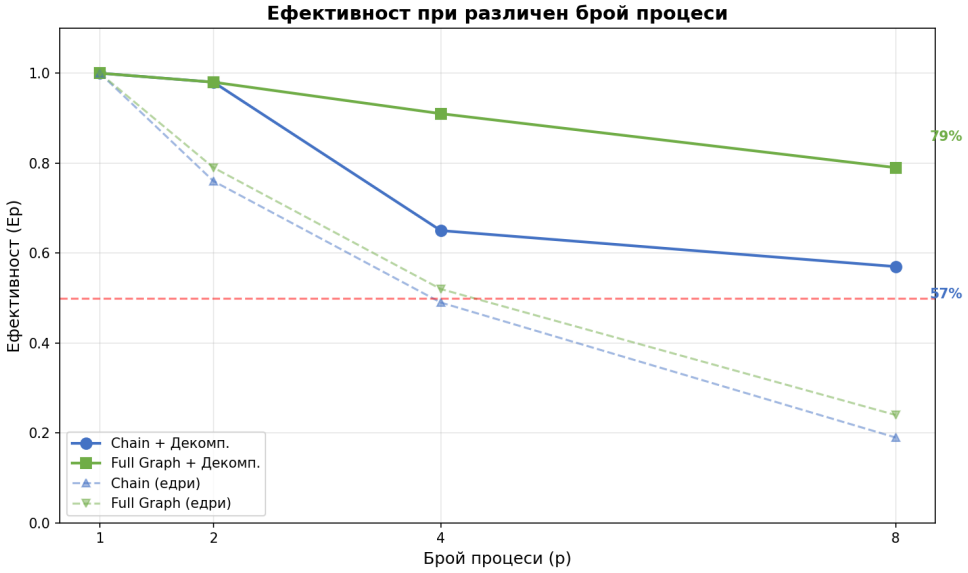
Фигура 1: Ускорение при различен брой процеси - декомпозицията драматично подобрява резултатите

8.3. Анализ на ефективността

Ефективността (Ep = Sp/p) показва колко добре се използват процесите:



Фигура 2: Ефективност при p=8 - декомпозицията постига 79% при Full Graph



Фигура 3: Ефективност при различен брой процеси

Ключови изводи от графиките:

- Без декомпозиция ефективността пада драматично при p=8 (14-30%)
- С декомпозиция Full Graph запазва **79% ефективност** при p=8
- Chain с декомпозиция достига **57% ефективност** при p=8
- Декомпозицията позволява истински паралелизъм - работата се разделя, не само задачите

8.4. Избор на оптимален праг

Прагът определя кога задачата се изчислява директно вместо да се декомпозира:

Топология	Оптимален праг	Тествани прагове	Ограничение
Full Graph	**30**	20-45	Няма - всички стабилни
Chain	**30**	20-45	Праг <30 блокира при p=1

Защо Chain блокира при нисък праг? При p=1 и праг<30 се генерират твърде много подзадачи, което препълва локалната опашка. Full Graph няма този проблем поради директната комуникация между всички процеси.

8.5. Сравнение на топологиите

Верижна топология (Chain):

- Всеки процес комуникира само с 2 съседа (ляв и десен)
- По-малко входящи заявки за обработка - добре за голям брой процеси
- **Недостатък:** Претоварен процес може да бъде достигнат само от съседите
- Подходяща за системи с локална (NUMA) топология

Пълен граф (Full Graph):

- Всеки процес може да комуникира директно с всеки друг
- **Предимство:** Директен достъп до претоварени процеси
- По-голям комуникационен overhead при много процеси
- Идеален за нехомогенно натоварване и малки клъстери

Ключов извод: При декомпозиция Full Graph е с **38% по-бърз** от Chain. Разликата идва от способността на всеки процес да краде работа директно от най-натоварения.

8.6. Обобщение на резултатите при p=8

Сценарий	Статично Sp	Chain Sp	Full Sp	Най-добър
Едри задачи (32)	1.15	1.49	1.89	Full (+64%)
Ситни задачи (128)	1.38	1.79	2.42	Full (+75%)
Декомпозиция fib(45)	-	4.58	**6.33**	Full (+38%)

9. Заключение

Проектът демонстрира предимствата на P2P архитектурите при разпределено динамично балансиране на паралелни приложения:

1. P2P Full Graph превъзхожда Chain топологията при нехомогенно натоварване:

- При едри задачи: Full е с 27% по-бърз от Chain (Sp=1.89 vs 1.49)
- При ситни задачи: Full е с 35% по-бърз от Chain (Sp=2.42 vs 1.79)
- При декомпозиция: Full е с **38% по-бърз** от Chain (Sp=6.33 vs 4.58)

2. Декомпозицията драматично подобрява ускорението:

- Full Graph: от Sp=1.89 до **Sp=6.33** (+235% подобрение)
- Chain: от Sp=1.49 до **Sp=4.58** (+207% подобрение)
- Декомпозицията позволява истински паралелизъм - работата се разделя, не само задачите

3. Изборът на праг е критичен:

- Оптимален праг за двете топологии: **30**
- Chain блокира при праг <30 (препълване на опашката при p=1)
- Full Graph работи стабилно при всички тествани прагове (20-45)

4. Гранулярността е критична за ефективността на work stealing:

- Едри задачи → ограничени възможности за преразпределение
- Ситни задачи → по-добро балансиране (+28% ускорение)
- Декомпозиция → най-добро балансиране (+235% ускорение)

5. Верижната топология е подходяща когато:

- Имаме голям брой процеси (комуникационен overhead)
- Хардуерът има NUMA топология
- Натоварването е относително равномерно

6. Пълният граф е по-добър при:

- Малък брой процеси (≤16)
- Силно нехомогенно натоварване
- Нужда от бързо преразпределение
- Използване на декомпозиция

Възможни подобрения

- Имплементация на **адаптивна топология**, която динамично се променя според натоварването

- **Йерархично балансиране** - локално в рамките на възел, глобално между възли
- Интеграция с **отказоустойчивост** - продължаване на работата при отказ на възел
- **Адаптивен праг** - автоматично определяне на оптималния праг според характеристиките на задачата

Принос на проекта

Проектът показва, че комбинацията от **P2P Full Graph топология** и **декомпозиция на задачите** постига най-високо ускорение (Sp=6.33 при 8 процеса, 79% ефективност). Това е значително подобрение спрямо традиционните подходи без декомпозиция.

10. Източници

1. Wilkinson, B., & Allen, M. (2004). *Parallel Programming: Techniques and Applications Using Networked Workstations and Parallel Computers*. Prentice Hall.
2. Gropp, W., Lusk, E., & Skjellum, A. (2014). *Using MPI: Portable Parallel Programming with the Message-Passing Interface*. MIT Press.
3. Kumar, V., Grama, A., Gupta, A., & Karypis, G. (2003). *Introduction to Parallel Computing*. Addison-Wesley.
4. Amazon Web Services. (2024). *Amazon EC2 Placement Groups*. AWS Documentation.
5. Blumofe, R. D., & Leiserson, C. E. (1999). Scheduling multithreaded computations by work stealing. *Journal of the ACM*, 46(5), 720-748.
6. Sanders, P., & Träff, J. L. (2002). The hierarchical factor algorithm for all-to-all communication. *Lecture Notes in Computer Science*.

Проектът е изготвен в рамките на курса по Разпределени Софтуерни Архитектури, 2026 г.